

Reviewer Pack — Bakry-Émery Curvature Floor Lower-Bounds Tseitin Resolution ...

Entry 6a507261ca55 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 19:42:19 UTC

Bakry-Émery Curvature Floor Lower-Bounds Tseitin Resolution Width

Entry ID: 6a507261ca55 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 19:42:19 UTC

1. Conjecture

Field A (mathematical branch): Discrete Bakry-Émery Γ_2 -curvature $CD(K, \infty)$ on finite graphs (Schmuckenschläger 1999; Lin-Yau 2010; Cushing-Liu-Münch 2019); the Bakry-Émery curvature $\kappa_{BE}(v; \infty)$ is the largest K such that $\Gamma_2(f, f)(v) \geq K \cdot \Gamma(f, f)(v)$ for all f , computed locally as the smallest generalized eigenvalue of an explicit symmetric matrix Q_v built from the 2-neighborhood of v . A diffusion-/stochastic-analysis invariant distinct from Ollivier-Ricci (Skenderi 2025, blacklisted) and Forman-Ricci (which collapses to a constant on 3-regular triangle-free graphs); rarely deployed in proof complexity. **Field B** (complexity object): Resolution refutation width $w(T(G, c) \vdash \perp)$ of unsatisfiable Tseitin formulas $T(G, c)$ on 3-regular graphs G with odd total F_2 -charge c , the Urquhart / Ben-Sasson-Wigderson / Alekhnovich-Razborov target. We use the lex-DPLL search-tree depth $d_{DPLL}(T(G, c))$ as a polynomially-tight proxy for w (Ben-Sasson-Wigderson).

Statement:

For every 3-regular graph G of girth ≥ 4 with odd-charge Tseitin instance $T(G, c)$, the average Bakry-Émery curvature $\bar{\kappa}(G) := (1/|V|) \sum_v \kappa_{BE}(v; \infty)$ is non-positive, and the resolution width obeys $w(T(G, c) \vdash \perp) \geq \lceil 0.4 \cdot |V(G)| \cdot (-\bar{\kappa}(G)) \rceil$. Equivalently, the rescaled invariant $\rho(G) := -\bar{\kappa}(G) \cdot |V(G)|$ is a Ben-Sasson-Wigderson-style width lower bound with multiplicative slack ≥ 0.4 across the cubic-girth- ≥ 4 family. A single 3-regular girth- ≥ 4 instance with $d_{DPLL} < 0.4 \cdot |V| \cdot (-\bar{\kappa}(G))$ refutes the conjecture.

Rationale (proposer’s reasoning):

Bakry-Émery curvature is a local Γ_2 obstruction quantifying how sharply diffusion concentrates near each vertex; on 3-regular triangle-free graphs it factors structurally through the 2-step walk pattern in a way that is provably finer than spectral gap (Cushing-Münch examples show $\bar{\kappa}$ separates graphs of equal λ_2). Tseitin width on cubic graphs is governed by edge-expansion (Ben-Sasson-Wigderson), and the discrete CD-Bonnet-Myers-type inequalities suggest $\bar{\kappa}$ controls a local ‘mass-redistribution cost’ that should match the bottleneck-counting argument behind Tseitin width. If true, this gives a coordinate-free local certificate for resolution width without invoking Cheeger or λ_2 explicitly.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6344657d6fe0473d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 72 instances (60 random 3-regular girth ≥ 4 + 12 dumbbell candidates), conjecture is SUPPORTED iff every instance satisfies $d_DPLL \geq 0.4 \cdot n \cdot (-\bar{\kappa}(G))$ with $\bar{\kappa}(G) \leq 0$ AND Pearson $r(d_DPLL/n, -\bar{\kappa}(G)) \geq 0.55$. A single violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Bakry-Emery curvature graph Tseitin resolution width - discrete Ricci curvature proof complexity expander Tseitin - CD(K, infinity) cubic graph resolution lower bound DPLL

Top relevant hits considered: - [http://arxiv.org/abs/2103.10093v1] Conjectures on Convergence and Scalar Curvature - [http://arxiv.org/abs/1807.10181v2] Gradient estimates, Bakry-Emery Ricci curvature and ellipticity for unbounded graph Laplacians - [http://arxiv.org/abs/1608.07778v2] Bakry-Emery curvature and diameter bounds on graphs - [http://arxiv.org/abs/2403.15973v1] Isoperimetric profile function comparisons with Integral Ricci curvature bounds - [http://arxiv.org/abs/1809.076] Discrete Ricci curvatures for directed networks - [http://arxiv.org/abs/1806.02734v3] Spectral lower bounds for the orthogonal and projective ranks of a graph - [http://arxiv.org/abs/1305.1190v2] Cubic symmetric graphs having an abelian automorphism group with two orbits - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=180.1s

5.1 Generated Python source

```
import random
import math
import sys
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3_regular_graph(n):
```

```

while True:
    edges = set()
    for i in range(n):
        neighbors = [j for j in range(n) if j != i]
        random.shuffle(neighbors)
        for j in neighbors[:2]:
            edge = tuple(sorted((i, j)))
            if (edge[1], edge[0]) not in edges:
                edges.add(edge)
    if len(edges) == n * 3 // 2 and all(len(set(v for u, v in edges if u == i)) == 2 for i in range(n)):
        return [set() for _ in range(n)], list(edges)

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i + 1, m):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def laplacian_matrix(G):
    n = len(G)
    L = [[0] * n for _ in range(n)]
    for i in range(n):
        degree = sum(1 for j in G[i] if (i, j) in edges or (j, i) in edges)
        L[i][i] = -degree
        for j in G[i]:
            if (i, j) in edges:
                L[i][j] = 1
    return L

def eigenvalues(A):
    n = len(A)
    A = [[A[i][j] for j in range(n)] for i in range(n)]
    Q = [[0] * n for _ in range(n)]
    R = [[0] * n for _ in range(n)]
    for k in range(n):
        Q[k][k] = 1
        r = [A[i][k] / A[k][k] if i != k else 0 for i in range(k + 1, n)]
        R[k] = r[:]
        for j in range(k + 1, n):

```

```

        A[j][k] -= sum(Q[j][i] * R[i][k] for i in range(k))
    Q = gaussian_elimination(Q)
    R = gaussian_elimination(R)
    eigenvals = [R[i][i] for i in range(n)]
    return eigenvals

def lex_dp11(T, assignment):
    if not T:
        return True
    var = next(v for v in range(len(T)) if v not in assignment)
    for val in (0, 1):
        new_assignment = assignment.copy()
        new_assignment[var] = val
        if all(new_assignment[v] == (T[v][i] ^ new_assignment[i]) % 2 for i in range(len(T[v])))
            if lex_dp11([t for t in T if var not in t], new_assignment):
                return True
    return False

def d_DPLL(T, c):
    assignment = [None] * len(T)
    return lex_dp11(T, assignment)

n = random.choice([8, 10, 12, 14, 16, 18, 20])
G, edges = generate_3_regular_graph(n)
L = laplacian_matrix(G)
eigenvals = sorted(eigenvalues(L))
kappa_BE = min(eigenvals[2:]) if len(eigenvals) > 2 else float('inf')
kappa_bar_G = kappa_BE / n
T = [(i, j) for j in G[i] if (i, j) in edges or (j, i) in edges] for i in range(n)]
c = random.choice([0, 1])
d_DPLL_val = d_DPLL(T, c)

return {
    "metric_name": "slack_ratio",
    "metric_value": d_DPLL_val / (n * (-kappa_bar_G)),
    "instances_tested": 1,
    "conjecture_holds": d_DPLL_val >= 0.4 * n * (-kappa_bar_G),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_slack_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_deviation = math.sqrt(sum((r["metric_value"] - mean_slack_ratio) ** 2 for r in results) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slack_ratio} std={std_deviation} support_fraction={su
    elif any(not r["conjecture_holds"] for r in results):

```

```

    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con"]
    print(f"RESULT: FALSIFIED counterexample=\"slack_ratio < 0.4\" first_failing_seed={first_f
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 180s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out after 180s (returncode 124) before producing any RESULT line or instance data, so neither the support nor falsification condition can be evaluated. Per the rules, a timeout yields INCONCLUSIVE. | next: Reduce the instance count or per-instance work (e.g., cap DPLL search depth, shrink to 24 instances, or precompute Bakry-Émery curvature) and rerun with a longer timeout to obtain the 72-instance metrics.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	364561
2	preregistration	claude_max	opus	0	0	6833
3	novelty	claude_max	opus	0	0	5811
4	novelty	claude_max	opus	0	0	7688
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22801
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15620
7	judge	claude_max	opus	0	0	4803

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 428117 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/6a507261ca55.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/6a507261ca55.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6a507261ca55.tar.gz` (if generated)